



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 3

по курсу «Алгоритмы компьютерной графики»

Студент группы ИУ9-42Б Федуков А. А.

Преподаватель: Цалкович П.А.

20 марта 2025 г.

Цель работы

- Определить параметризованную модель объекта сцены (в соответствии с вариантом).
- Определить преобразования, позволяющие получить заданный вид проекции (в соответствии с вариантом). Для демонстрации проекции добавить в сцену куб (в стандартной ориентации, не изменяемой при модельно-видовых преобразованиях основного объекта). <
- Реализовать изменение ориентации и размеров объекта (навигацию камеры) с помощью модельно-видовых преобразований (без `gluLookAt`). Управление производится интерактивно с помощью клавиатуры и/или мыши.
- Предусмотреть возможность переключения между каркасным и твердотельным отображением модели (`glFrontFace` / `glPolygonMode`).

Мой вариант заключался в реализации изометрической проекции эллиптического тора

Теория

В ходе выполнения данной лабораторной работы основное внимание было уделено построению эллиптического тора и его изометрической проекции. Основой работы послужили параметры тора, описывающие его форму через два радиуса: большой радиус, определяющий расстояние от центра тора до оси вращения, и малый радиус, задающий толщину трубы, из которой состоит тор. Для построения тора использовалось параметрическое уравнение, позволяющее рассчитать каждую точку поверхности в зависимости от двух угловых переменных.

Изометрическая проекция, применённая к объекту, представляет собой тип параллельной проекции, которая исключает перспективные искажения. Основным методом заключался в последовательных поворотах объекта на 45 градусов вокруг оси Y и на 35.264 градуса вокруг оси X, что позволяло достичь равномерного отображения всех сторон. Эти углы позволяют представить трёхмерные

объекты таким образом, что их стороны выглядят равными независимо от их удалённости от камеры.

При выполнении задания были использованы методы работы с матрицами трансформаций, такие как вращение, масштабирование и перенос. Эти матрицы были применены для изменения положения, ориентации и размеров тора, что позволило настраивать параметры отображаемого объекта в реальном времени. Кроме того, было реализовано переключение между каркасным и твердотельным режимами отрисовки, что дало возможность визуализировать геометрию объекта различными способами.

Реализация

Используя `ruopengl` я описал тор при помощи параметрического уравнения, а затем рисовал один статическим, а другой, откликающийся на нажатия клавиш. Потом я применил к ним изометрическую проекцию.

Код

Листинг 1: Файл `main.py`

```
1 import glfw
2 from OpenGL.GL import *
3 from math import cos, sin, pi
4
5 # Parameters for rotation, scaling, translation
6 angle_x, angle_y = 0, 0
7 scaleVec = [0.5, 0.5, 0.5]
8 translateVec = [0, 0, 0]
9 translateRatio = 0.1
10 wireframe = False # Mode switch
11
12
13 # Rotation matrix for isometric projection
14 def apply_isometric_projection():
15     glRotatef(35.264, 1, 0, 0) # Rotate 35.264 degrees along X-axis
16     glRotatef(45, 0, 1, 0)    # Rotate 45 degrees along Y-axis
17
18
19 def main():
20     if not glfw.init():
```

```

21         return
22     window = glfw.create_window(800, 800, "Isometric Cube", None, None)
23     if not window:
24         glfw.terminate()
25         return
26     glfw.make_context_current(window)
27     glfw.set_key_callback(window, key_callback)
28     glfw.set_scroll_callback(window, scroll_callback)
29
30     glEnable(GL_DEPTH_TEST)
31
32     while not glfw.window_should_close(window):
33         display(window)
34     glfw.destroy_window(window)
35     glfw.terminate()
36
37 # Функция для создания эллиптического тора
38 def draw_elliptical_torus(R, r, num_major, num_minor):
39     for i in range(num_major):
40         theta1 = i * 2 * pi / num_major
41         theta2 = (i + 1) * 2 * pi / num_major
42
43         glBegin(GL_QUAD_STRIP)
44         for j in range(num_minor + 1):
45             phi = j * 2 * pi / num_minor
46             for theta in [theta1, theta2]:
47                 x = (R + r * cos(phi)) * cos(theta)
48                 y = (R + r * cos(phi)) * sin(theta) * 1.8
49                 z = r * sin(phi)
50                 glVertex3f(x, y, z)
51         glEnd()
52
53
54 def display(window):
55     global angle_x, angle_y
56     glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT)
57     # glClearColor(1.0, 1.0, 1.0, 1.0)
58
59     # Draw static cube
60     glLoadIdentity()
61     apply_isometric_projection() # Apply isometric projection
62     glTranslatef(1, 0, 0)        # Move static cube to the left
63     glScalef(*scaleVec)
64     # Отрисовка эллиптического тора
65     glColor3f(0.2, 0.7, 1.0)
66     draw_elliptical_torus(1.0, 0.3, 40, 20)

```

```

67
68 # Draw rotating cube
69 # Apply transformations
70 glLoadIdentity()
71 glTranslatef(*translateVec)
72 glScalef(*scaleVec)
73 apply_isometric_projection() # Apply isometric projection
74
75 glRotatef(angle_x, 1, 0, 0) # Rotate along X-axis
76 glRotatef(angle_y, 0, 1, 0) # Rotate along Y-axis
77 # Отрисовка эллиптического тора
78 glColor3f(0.8, 0.1, 1.0)
79 draw_elliptical_torus(1.0, 0.3, 40, 20)
80
81 glfw.swap_buffers(window)
82 glfw.poll_events()
83
84
85 def key_callback(window, key, scancode, action, mods):
86     global angle_x, angle_y
87     global translateVec, scaleVec, wireframe
88
89     if action == glfw.PRESS or action == glfw.REPEAT:
90         # Cube rotation
91         if key == glfw.KEY_RIGHT:
92             angle_y -= 3
93         if key == glfw.KEY_LEFT:
94             angle_y += 3
95         if key == glfw.KEY_UP:
96             angle_x -= 3
97         if key == glfw.KEY_DOWN:
98             angle_x += 3
99
100         # Translation (Move by X, Y, Z)
101         if key == glfw.KEY_W:
102             translateVec[1] += translateRatio
103         if key == glfw.KEY_S:
104             translateVec[1] -= translateRatio
105         if key == glfw.KEY_A:
106             translateVec[0] -= translateRatio
107         if key == glfw.KEY_D:
108             translateVec[0] += translateRatio
109         if key == glfw.KEY_Q:
110             translateVec[2] += translateRatio # Move forward
111         if key == glfw.KEY_E:
112             translateVec[2] -= translateRatio # Move backward

```

```

113
114     # Scaling
115     if key == glfw.KEY_Z:
116         scaleVec[0] += 0.1
117         scaleVec[1] += 0.1
118         scaleVec[2] += 0.1
119     if key == glfw.KEY_X:
120         scaleVec[0] -= 0.1
121         scaleVec[1] -= 0.1
122         scaleVec[2] -= 0.1
123
124     # Toggle between wireframe and solid
125     if key == glfw.KEY_M:
126         wireframe = not wireframe
127         if wireframe:
128             glPolygonMode(GL_FRONT_AND_BACK, GL_LINE)
129         else:
130             glPolygonMode(GL_FRONT_AND_BACK, GL_FILL)
131
132     # Close window
133     if key == glfw.KEY_ESCAPE:
134         glfw.set_window_should_close(window, 1)
135
136
137 def scroll_callback(window, xoffset, yoffset):
138     global scaleVec
139     scaleVec[0] += yoffset * 0.1
140     scaleVec[1] += yoffset * 0.1
141     scaleVec[2] += yoffset * 0.1
142
143
144 if __name__ == "__main__":
145     main()

```

Вывод программы

Программа создала окно и отрисовала два тора в изометрической проекции. Синий всегда был неподвижен, а розовый можно было вращать и переключать режим отображения на каркасный.

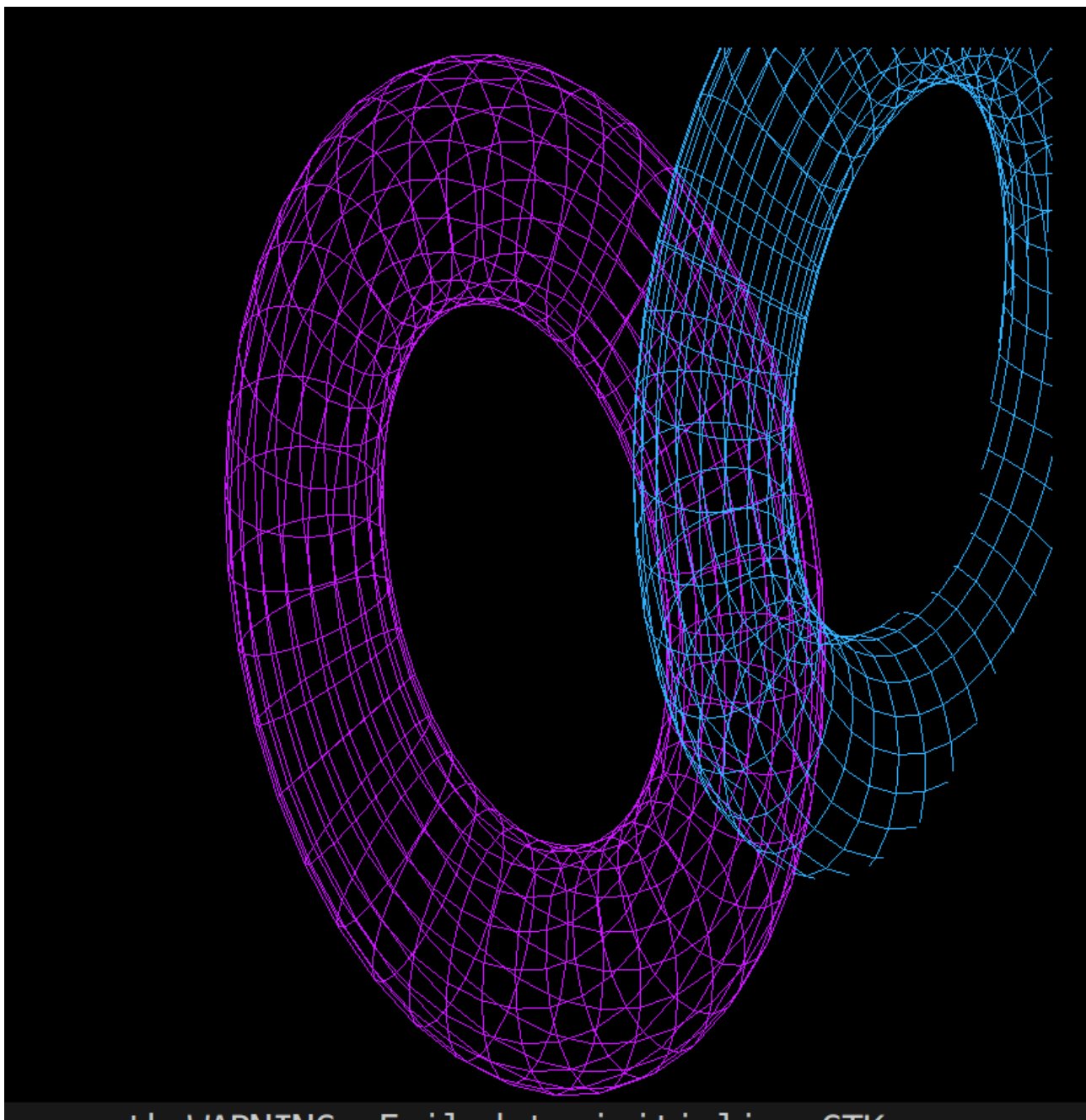


Рис. 1 — Исходная фигура

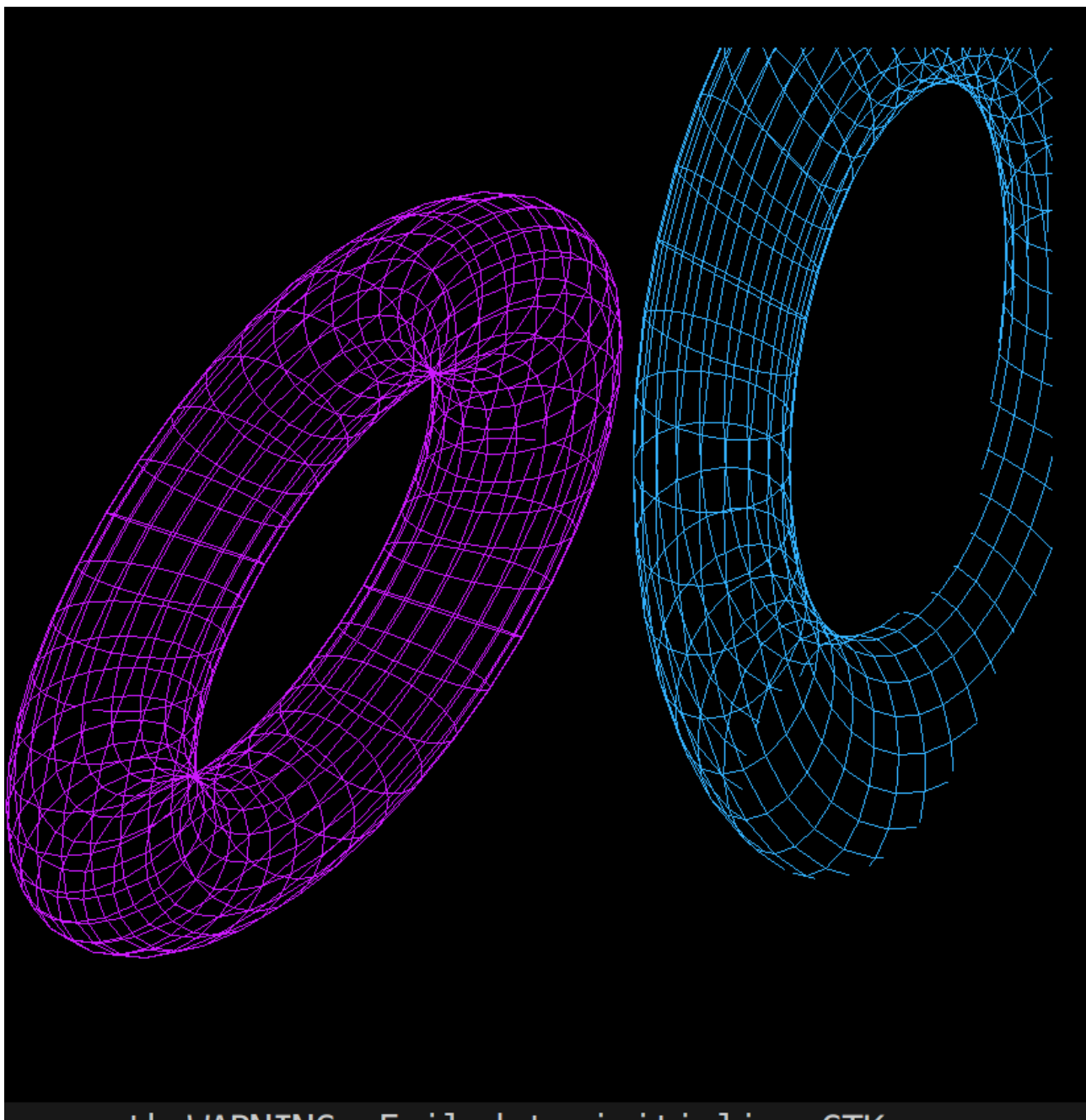


Рис. 2 — Фигура после перемещения

Вывод

В ходе лабораторной работы мною было продолжено изучены основ работы с OpenGL. Была выполнена задача графического отображения эллиптического тора в изометрической проекция, а также управление его модельно-видовыми преобразованиями и переключение между каркасным и твердотельным режимами отображения.